

Linux Embarqué

Cours

Florian Bescher
ISEN Ouest – M1 2025/2026

Linux Embarqué

- **Partie 1 : Linux Embarqué**

Enseignant : Florian Bescher

Cours : 4h30, classe entière

TP : 4 × 3h, demi-groupe

- **Partie 2 : Programmation Système**

Enseignante : Audrey Queudet

Cours + TP : 3 × 3h, demi-groupe

- Un DS commun (durée 1h30)

Prérequis

- Aisance à la ligne de commande Linux
- Bases de programmation en C
- Connaissances de base des périphériques embarqués (GPIO, PWM, ADC)

Le monde de l'embarqué

Définition

Système embarqué

Système électronique, piloté par un logiciel **intégré** au système qu'il contrôle, et répondant à un certains nombre de contraintes externes.

Contraintes

- Encombrement
- Coût
- Consommation
- Sureté et sécurité
- Environnement (température, pression, radiation, ...)

Applications

- Transport : automobile, ferroviaire, avionique...
- Spatial : fusée, rover, satellites...
- Militaire : drone, missiles, radar...
- Industrie : robot industriel, automates...
- Électroménager : lave-linge, frigo, thermostat...

Marché et industrie

Objectifs

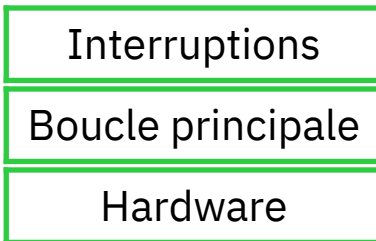
- *Time to Market* : réduire les temps de développement et de test/qualification pour mettre le produit rapidement sur le marché
- *Design reuse* : réutiliser des éléments existants
- Disponibilité du support technique

Le monde de l'embarqué

Catégories

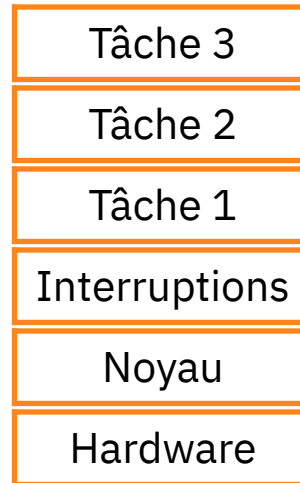
Architecture simple (microcontrôleur)

STM32, Arduino



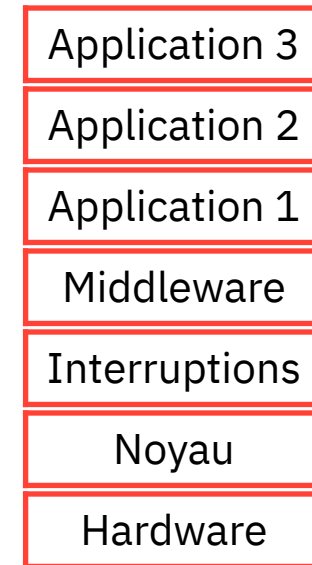
Architecture intermédiaire

FreeRTOS



Architecture complexe

Basée sur un OS type Linux



GNU/Linux

Historique

Noyau Linux

- Créé en 1991 par Linux Torvalds
- Libre de droits (license GPLv2.0)

GNU

- Projet fondé par Richard Stallman
- Ensemble d'utilitaires systèmes (`ls`, `mkdir`, `cd` ...)
- Libre de droits (license GPLv3.0)



Fig. 1. – Linus Torvalds

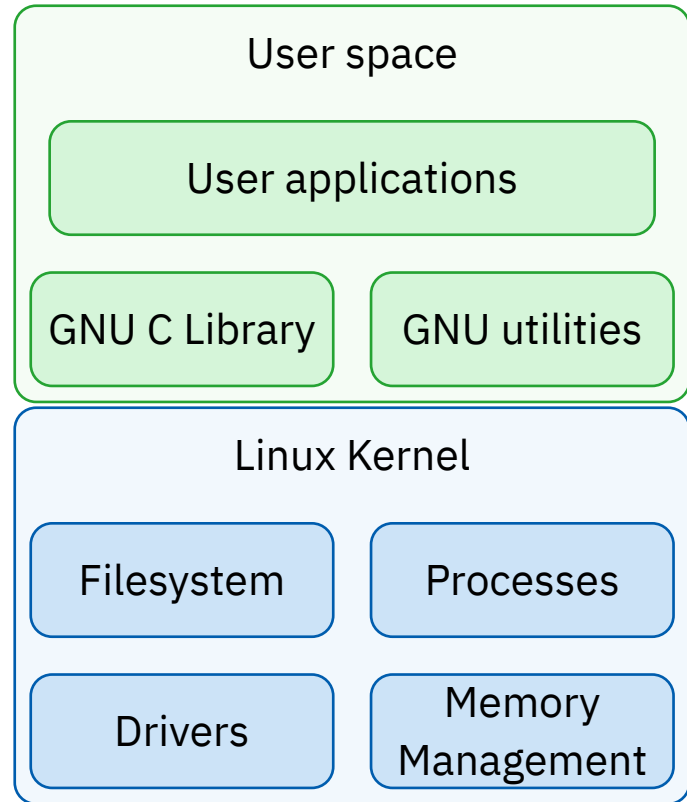
Krd & Von Sprat · CC-BY-SA 4.0



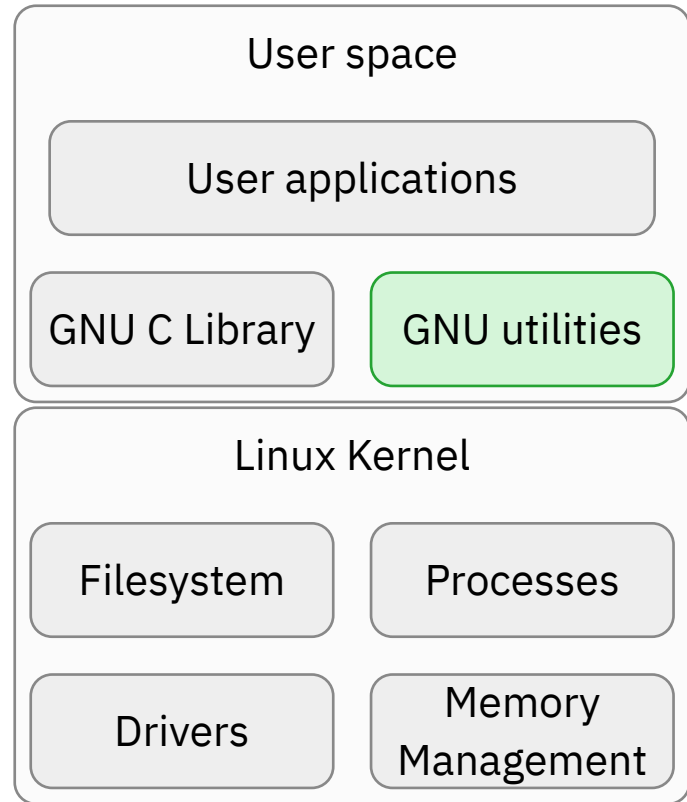
Fig. 2. – Richard Stallman

NicoBZH · CC-BY-SA 2.0

Composantes du système



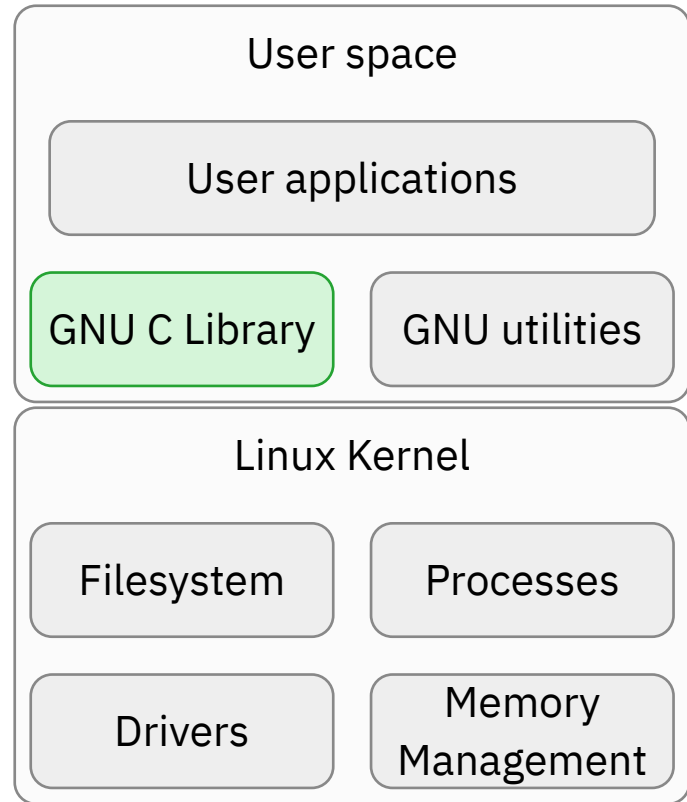
Composantes du système



GNU utilities

- ls
- cd
- mkdir
- touch
- ...

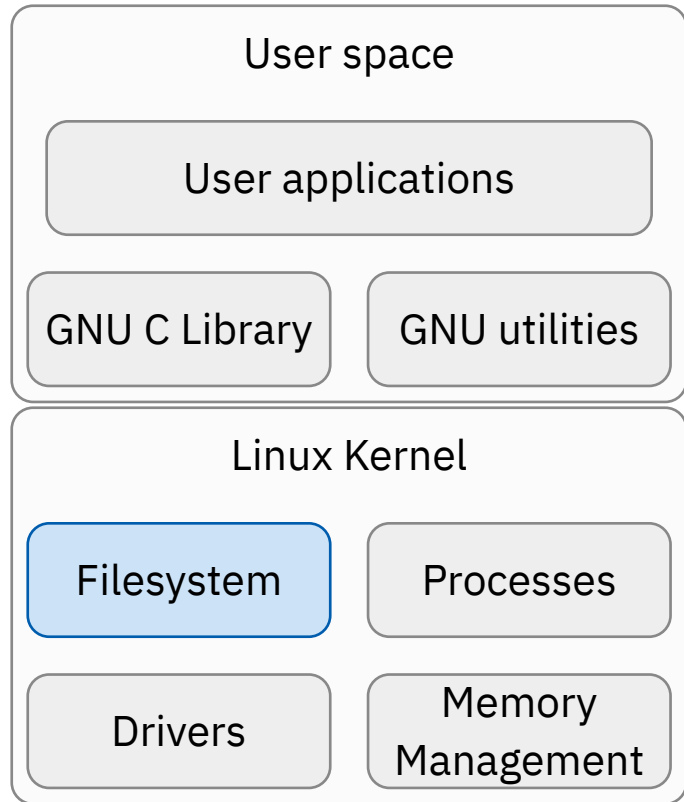
Composantes du système



GNU C Library

- `fopen`, `fclose`, `fprintf`, ...
- `scanf`, `printf`, ...
- `malloc`, `free`, ...

Composantes du système



Linux filesystem

```
/
├── boot          # Lancement du système
├── dev           # Périphériques (devices)
├── etc           # Configuration
├── home         # Documents personnels
├── proc         # Informations système
├── sys          # Composants matériels
├── tmp          # Fichiers temporaires
├── usr
│   ├── bin      # Applications
│   └── lib      # Bibliothèques
└── var         # Données système
```

Linux dans l'embarqué

Avantages

- Fiabilité de fonctionnement
- Sécurité de fonctionnement
- Portabilité (fonctionne sur de nombreuses plateformes)
- Diversité des utilitaires et application
- Libre de droits
- Support de la communauté

Mais inadapté dans les cas suivants

- Applications à ultra faible consommation
- Applications à très bas coût
- Si le dispositif est très limité en mémoire

Architectures processeur

Processeur et assembleur

Architecture CPU

Ensemble des instructions qu'un processeur peut réaliser.



Fig. 3. – Intel Core i5

Ashley Pomeroy · CC-BY-SA 4.0

x86_64



Fig. 4. – Apple Silicon M1

Sonic8400 · CC-BY-SA 4.0

aarch64

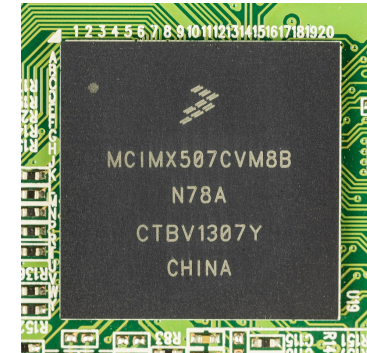


Fig. 5. – ARM Cortex A8

Raimond Spekking · CC-BY-SA 4.0

ARM v7

Processeur et assembleur

Architecture CPU

Ensemble des instructions qu'un processeur peut réaliser.

```
push    rbp
mov     rbp, rsp
mov     DWORD PTR [rbp-4], edi
mov     eax, DWORD PTR [rbp-4]
imul    eax, eax
pop     rbp
ret
```

x86_64

```
sub     sp, sp, #16
str     w0, [sp, 12]
ldr     w0, [sp, 12]
mul     w0, w0, w0
add     sp, sp, 16
ret
```

aarch64

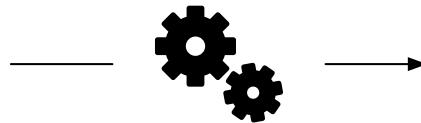
```
str     fp, [sp, #-4]!
add     fp, sp, #0
sub     sp, sp, #12
str     r0, [fp, #-8]
ldr     r3, [fp, #-8]
ldr     r2, [fp, #-8]
mul     r1, r2, r3
mov     r3, r1
mov     r0, r3
add     sp, fp, #0
ldr     fp, [sp], #4
bx      lr
```

ARM v7

Cross-compilation

Objectif : compiler depuis une architecture A vers une architecture B

```
int square(int a) {  
    return a * a;  
}
```



```
str    fp, [sp, #-4]!  
add    fp, sp, #0  
sub    sp, sp, #12  
str    r0, [fp, #-8]  
ldr    r3, [fp, #-8]  
ldr    r2, [fp, #-8]  
mul    r1, r2, r3  
mov    r3, r1  
mov    r0, r3  
add    sp, fp, #0  
ldr    fp, [sp], #4  
bx     lr
```

Développé et compilé sur x86_64 pour ARM v7

Carte Beaglebone Black

Carte Beaglebone Black

La carte

- Conçue par la fondation Beaglebone
- Open Hardware



Fig. 6. – Carte Beaglebone Black

Principaux composants

- Processeur Sitara AM3358BZCZ100
Architecture ARM v8
- RAM : 512 Mo
- EEPROM : 4 ko
- eMMC : 4 Go
- Connectivité : Ethernet, HDMI, USB, carte SD
- Interfaces : GPIO, PWM, ADC, I2C, SPI

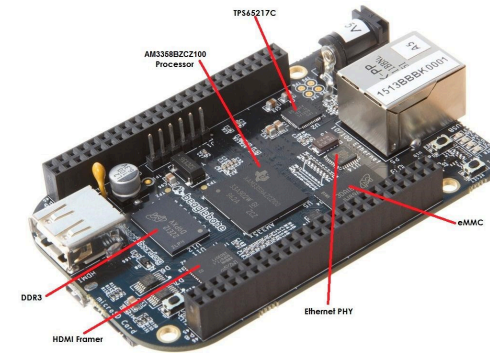


Fig. 7. – Principaux composants